Unit -3
Plotting using PyLab

Plotting using PyLab

- PyLab is a module in Python that combines the functionalities of NumPy and Matplotlib. It provides a simple interface for creating graphs and visualizing data in a MATLAB-like environment.

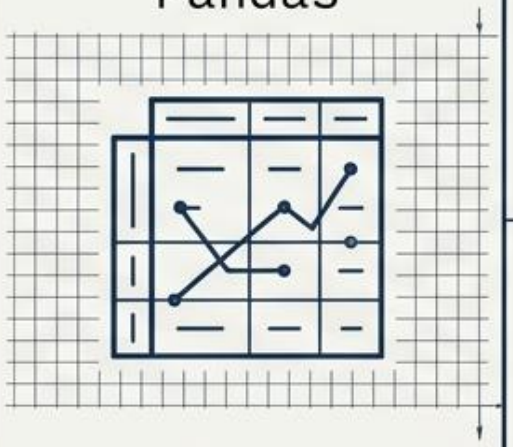
- **Steps for Plotting Using PyLab**

1. Import the PyLab module.
2. Create the data for the X-axis and Y-axis.
3. Use the plot() function to draw the graph.
4. Add a title and axis labels.
5. Display the graph using show().

The Python Scientific Architecture

Manipulation

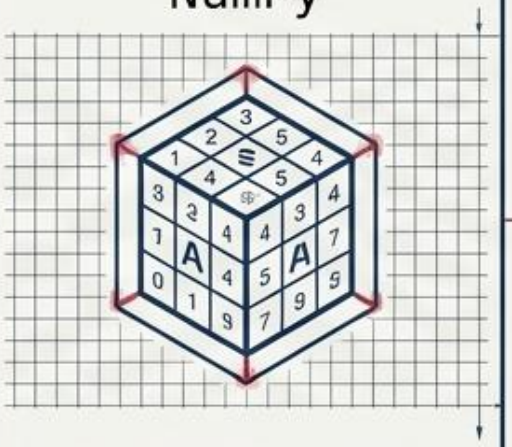
Pandas



Extracts and cleans raw CSV data.

Computation

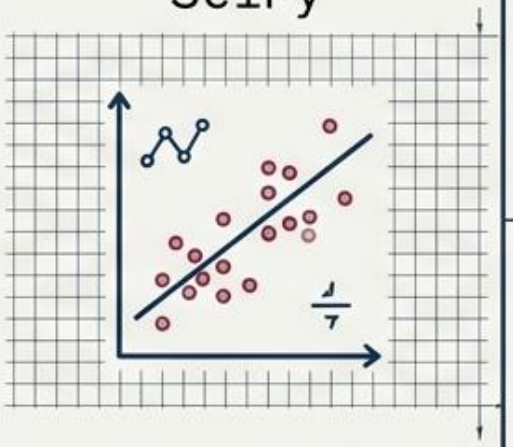
NumPy



Handles N-dimensional arrays and fast numerical operations.

Modeling

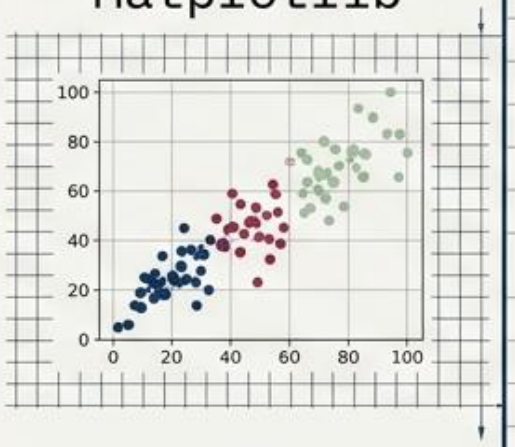
SciPy



Executes statistical modeling and uncertainty quantification.

Visualization

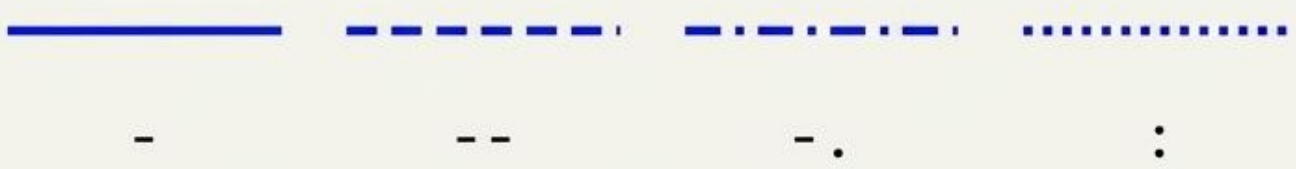
Matplotlib



The canvas that brings the numbers to life.

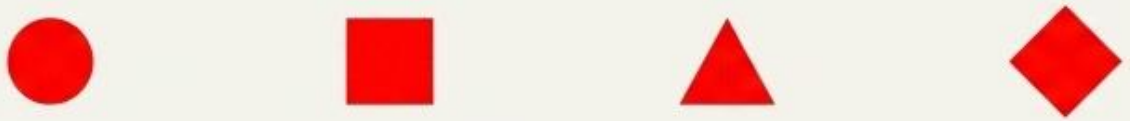
Styling the Canvas: Lines and Markers

Line Styles (linestyle)



`-` `--` `-.` `:`

Markers (marker)



`o` `s` `^` `D`

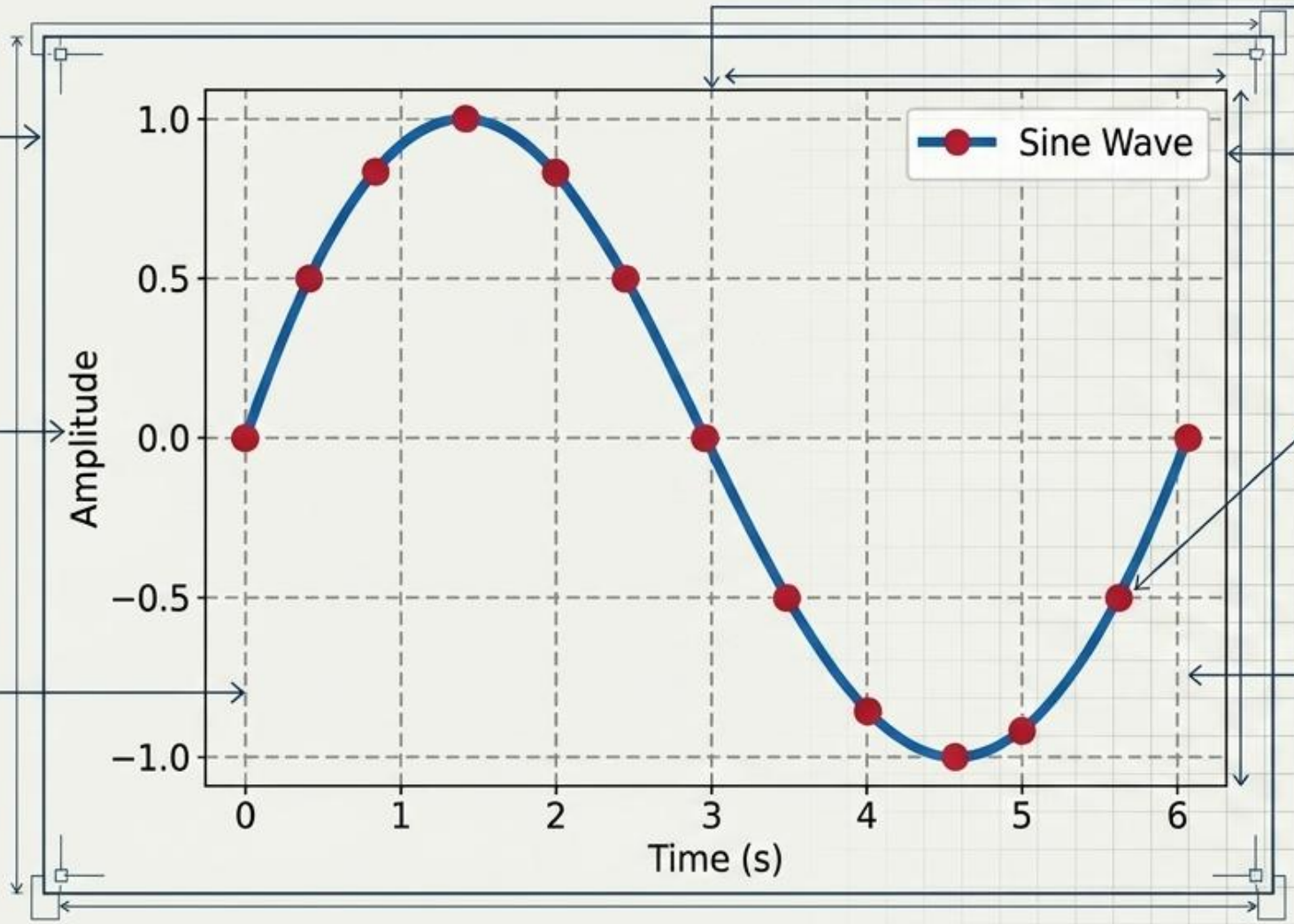
Colors



`b` `g` `r` `c` `m` `y` `k` `w`

```
plt.plot(x, y,
    color='green',
    linestyle='--',
    marker='o',
    markerfacecolor='red')
```

Anatomy of a Publication-Quality Plot



`Figure`: The overall canvas containing everything.

`Axes`: The actual plot area.

`Spines`: The data boundaries.

`Spines`: The data boundaries.

`Legend`: The key explaining the plotted series.

`Markers`: The specific data point symbols.

`Gridlines`: The background reference mesh.

Example 1: Simple Line Graph

```
from pylab import *

x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

plot(x, y)
title("Simple Line Plot")
xlabel("X-Axis")
ylabel("Y-Axis")
show()
```

Function	Description
plot()	Draws a line graph
scatter()	Creates a scatter plot
bar()	Creates a bar chart
hist()	Draws a histogram
pie()	Creates a pie chart
xlabel()	Sets the X-axis label
ylabel()	Sets the Y-axis label
title()	Adds a graph title
grid()	Displays grid lines
legend()	Shows graph legend
show()	Displays the graph

Advantages of PyLab

- Easy to create graphs and charts.
- Supports various plot types like line, bar, scatter, and histogram.
- Useful for scientific computing and data visualization.
- Works seamlessly with NumPy arrays.
- Provides a MATLAB-like programming environment.

Conclusion

- PyLab is a powerful Python module used for **data visualization**.
- It helps users represent numerical data graphically, making it easier to analyze trends, compare values, and understand relationships between datasets.

Plotting Mortgages and Extended Examples (PyLab)

Definition

- A **mortgage** is a loan used to purchase property, where the borrower repays the loan in fixed monthly installments over a specified period. Using **PyLab**, mortgage payments and loan balances can be visualized graphically, making it easier to analyze repayment trends.

Example 1: Plotting Monthly Mortgage Payments

```
from pylab import *

months = [1, 2, 3, 4, 5, 6]
payments = [15000, 15000, 16000, 15000, 20000, 15000]

plot(months, payments, 'b-o')
title("Monthly Mortgage Payments")
xlabel("Months")
ylabel("Payment Amount (₹)")
grid(True)
show()
```

Extended Example: Comparing Two Mortgage Plans

```
from pylab import *
months = [1, 2, 3, 4, 5, 6]
planA = [1000000, 980000, 960000, 940000, 920000, 900000]
planB = [1000000, 985000, 970000, 955000, 940000, 925000]
plot(months, planA, label="Plan A", color="blue")
plot(months, planB, label="Plan B", color="green")
title("Comparison of Mortgage Plans")
xlabel("Months")
ylabel("Remaining Balance (₹)")
legend()
grid(True)
show()
```

Applications of Mortgage Plotting

- Compare different loan repayment plans.
- Analyze monthly payment trends.
- Monitor remaining loan balance.
- Estimate total repayment over time.
- Support financial planning and decision-making.

- **Advantages**
- Provides a clear visual representation of loan repayment.
- Makes it easy to compare multiple mortgage plans.
- Helps identify long-term repayment patterns.
- Useful for banks, financial institutions, and borrowers.
- **Conclusion**
- Plotting mortgages with **PyLab** helps visualize **monthly payments**, **remaining loan balance**, and **comparisons between mortgage plans**. These graphs simplify financial analysis and assist borrowers in selecting the most suitable mortgage option.

Fibonacci Sequence Revisited (Dynamic Programming) Definition

- **Definition**
- The Fibonacci sequence is a series of numbers in which each number is the sum of the two preceding numbers.
- $F(0)=0, \quad F(1)=1$
- $F(n)=F(n-1)+F(n-2), n \geq 2$
- The sequence is:0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Dynamic Programming Approach

- **Dynamic Programming (DP)** is an optimization technique that stores previously computed Fibonacci numbers to avoid repeated calculations. This makes the algorithm much faster than simple recursion.
- **Algorithm**
- Initialize the first two Fibonacci numbers.
- Store them in a list.
- Compute each new Fibonacci number using the previous two values.
- Return the required Fibonacci number.

Dynamic Programming with Less Memory

```
def fibonacci(n):
    if n <= 1:
        return n

    a, b = 0, 1

    for i in range(2, n + 1):
        a, b = b, a + b

    return b

print(fibonacci(10))
```

Time and Space Complexity

Method	**Time Complexity**	**Space Complexity**
Simple Recursion	$O(2^n)$	O(n)
Dynamic Programming (Array)	O(n)	O(n)
Dynamic Programming (Optimized)	O(n)	O(1)

- **Advantages of Dynamic Programming**
- Eliminates repeated calculations.
- Faster than recursive implementation.
- Efficient for large values of **n**.
- Improves overall program performance.

- **Conclusion**
- The **Dynamic Programming** approach to the Fibonacci sequence stores previously calculated values, reducing redundant computations. It computes the nth Fibonacci number in **O(n)** time, making it much more efficient than the traditional recursive method.

0/1 Knapsack Algorithm

The 0/1 Knapsack

- The 0/1 Knapsack Problem states that you have a backpack with a weight limit, and you are in a room full of treasures, each treasure with a value and a weight.
- To solve the 0/1 Knapsack Problem you must figure out which treasures to pack to maximize the total value, and at the same time keeping below the backpack's weight limit.



Microscope
$ 300
2 kg



Globe
$ 200
1 kg



Cup
$ 400
5 kg



Crown
$ 500
3 kg



Knapsack
$ 0
0/10 kg

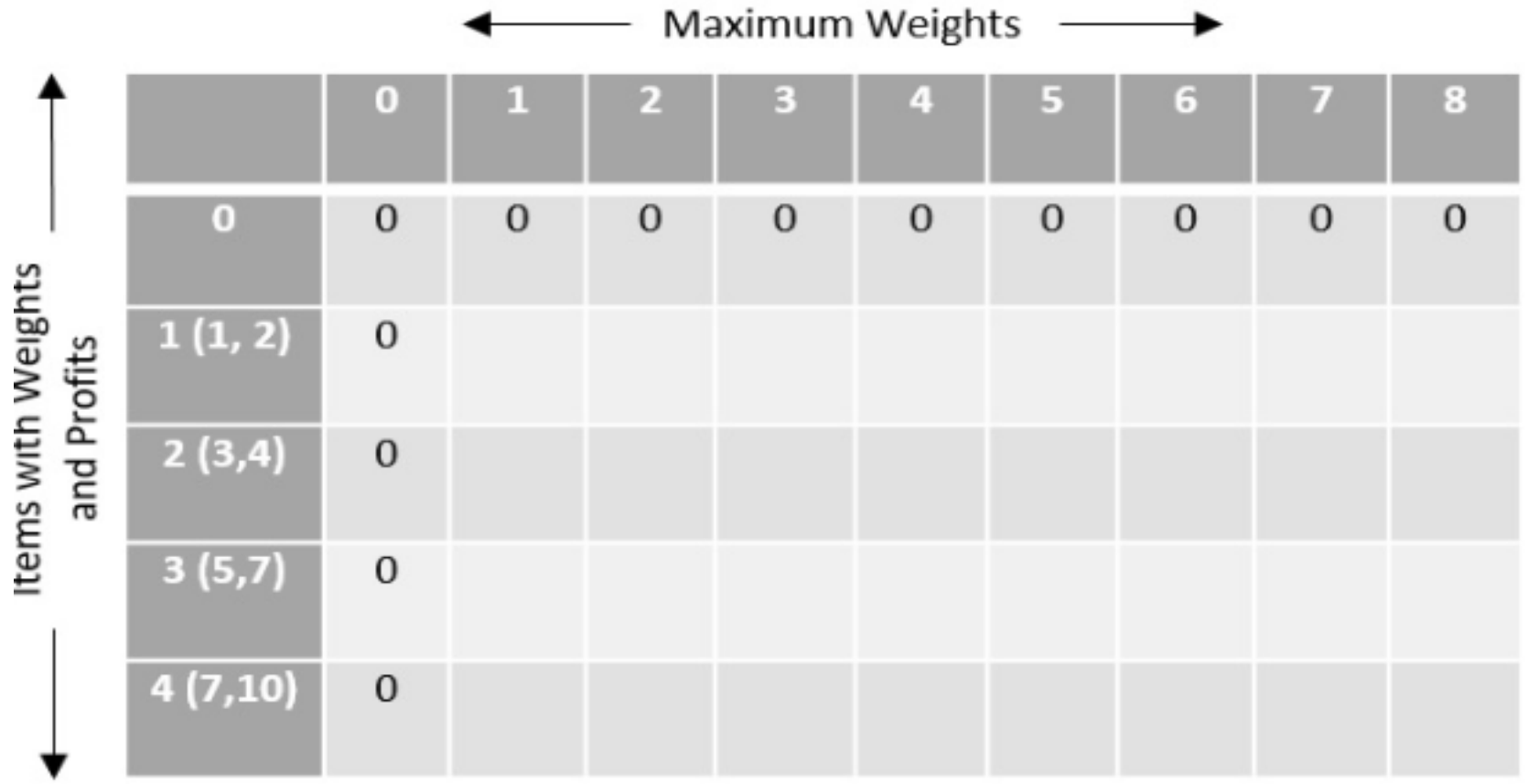
- Solving the 0/1 Knapsack Problem helps businesses decide which projects to fund within a budget, maximizing profit without overspending. It is also used in logistics to optimize the loading of goods into trucks and planes, ensuring the most valuable, or highest prioritized, items are included without exceeding weight limits.
- **The 0/1 Knapsack Problem**
- Rules:
- Every item has a weight and value.
- Your knapsack has a weight limit.
- Choose which items you want to bring with you in the knapsack.
- You can either take an item or not, you cannot take half of an item for example.
- Goal:
- Maximize the total value of the items in the knapsack.

Let us consider that the capacity of the knapsack is W = 8 and the items are as shown in the following table.

Item	A	B	C	D
Profit	2	4	7	10
Weight	1	3	5	7

Step 1

Construct an adjacency table with maximum weight of knapsack as rows and items with respective weights and profits as columns.



	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 (1, 2)	0								
2 (3,4)	0								
3 (5,7)	0								
4 (7,10)	0								

The formula to store the profit values is −

$$c[i,w]=\max\{c[i-1,w-w[i]]+P[i]\}$$

By computing all the values using the formula, the table obtained would be −

Maximum Weights →

Items with Weights and Profits ↓

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 (1, 2)	0	1	1	1	1	1	1	1	1
2 (3,4)	0	1	1	4	6	6	6	6	6
3 (5,7)	0	1	1	4	6	7	9	9	11
4 (7,10)	0	1	1	4	6	7	9	10	12

- To find the items to be added in the knapsack, recognize the maximum profit from the table and identify the items that make up the profit, in this example, its {1, 7}.

Maximum Weights →

Items with Weights and Profits ↓

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 (1, 2)	0	1	1	1	1	1	1	1	1
2 (3,4)	0	1	1	4	6	6	6	6	6
3 (5,7)	0	1	1	4	6	7	9	9	11
4 (7,10)	0	1	1	4	6	7	9	10	(12)

```
def knapsack(W, wt, val, n):
  K = [[0] * (W+1) for i in range (n+1)]
  for i in range(n+1):
    for w in range(W+1):
      if(i == 0 or w == 0):
        K[i][w] = 0
      elif(wt[i-1] <= w):
        K[i][w] = max(val[i-1] + K[i-1][w-wt[i-1]], K[i-1][w])
      else:
        K[i][w] = K[i-1][w]
  return K[n][W]

val = [2,4,7,10]
wt = [1, 3, 5, 7]
W = 8
ln = len(val)
profit = knapsack(W, wt, val, ln)
print("Maximum Profit achieved with this knapsack: ")
print(profit)
```

DYNAMIC PROGRAMMING AND DIVIDE AND CONQUER

- Like divide-and-conquer algorithms, dynamic programming is based upon solving independent subproblems and then combining those solutions.
- There are, however, some important differences.
- Divide-and-conquer algorithms are based upon finding subproblems that are substantially smaller than the original problem.

- For example, merge sort works by dividing the problem size in half at each step. In contrast, dynamic programming involves solving problems that are only slightly smaller than the original problem. For example, computing the 19th Fibonacci number is not a substantially smaller problem than computing the 20th Fibonacci number.
- Another important distinction is that the efficiency of divide-and-conquer algorithms does not depend upon structuring the algorithm so that the same problems are solved repeatedly.
- In contrast, dynamic programming is efficient only when the number of distinct subproblems is significantly smaller than the total number of subproblems.